

MATHEMATICAL MODELING & SIMULATION

REPORT OF THE INTERNSHIP PROJECT

Submitted to the
ST. ALOYSIUS COLLEGE , ELTHURUTH

In partial fulfilment of the requirements for the award of the degree
FOUR YEAR UNDER GRADUATE PROGRAMME (FYUGP)
BSc. Physics (Honours)

By
ABHISHEK P G
ELAYSPH001

Title of the Project
**Heat Diffusion in One Dimension:
From Physical Intuition to Numerical Simulation**

Under the guidance of
Dr. Vijitha Mukundan
Department of Mathematics
Sacred Heart College (Autonomous), Chalakudy

Postgraduate & Research Department of Mathematics
SACRED HEART COLLEGE (AUTONOMOUS), CHALAKUDY



Department of Physics
St. Aloysius College, Elthuruth, Thrissur
MAY, 2026

DECLARATION

I hereby declare that the Internship entitled “**Heat Diffusion in One Dimension: From Physical Intuition to Numerical Simulation**” is an authentic work done by me under the guidance and supervision of **Dr. Vijitha Mukundan, Sacred Heart College (Autonomous), Chalakudy**. I hereby submit the Internship report for the partial fulfilment of requirement for award of Degree of Four Year Degree Programme, BSc. Physics (Honours) in Physics.

Signature of Student _____

Name of student: ABHISHEK P G

Register no: ELAYSPH001

Signature of Internship Supervisor/ Mentor: _____

Chalakudy
May 2026

CERTIFICATE

This is to certify that the Internship work entitled “**Heat Diffusion in One Dimension: From Physical Intuition to Numerical Simulation**” is bonafide work done by **ABHISHEK P G, ELAYSPH001** under the guidance of **Dr. Vijitha Mukundan, Sacred Heart College (Autonomous), Chalakudy**, submitted for partial fulfilment of requirement of award of Degree of Four Year Degree Programme, BSc. Physics (Honours) in Physics. The internship was of **60 Hours** duration in the period **28 April 2026 to 02 May 2026 and 25 May 2026 to 29 May 2026 (10 days)**.

Signature and Seal of IPO

ACKNOWLEDGEMENT

I express my deepest sense of gratitude to **Dr. Vijitha Mukundan**, Assistant Professor, Postgraduate & Research Department of Mathematics, **Sacred Heart College (Autonomous), Chalakudy**, for her valuable guidance, constant encouragement, and support throughout the period of this internship.

I am also thankful to the Head of the Department and all other faculty members of the Department of Physics, **St. Aloysius College (Autonomous), Elthuruth, Thrissur**, for their help and cooperation during my studies.

I extend my sincere thanks to the authorities of **Sacred Heart College (Autonomous), Chalakudy**, for providing me with the necessary facilities and a conducive environment to complete my internship project on “**Heat Diffusion in One Dimension: From Physical Intuition to Numerical Simulation**”.

Finally, I would like to thank my parents and friends for their continuous support and prayers which helped me to complete this work successfully.

I also acknowledge the use of **Claude** (Anthropic, 2025), an AI language model, for assistance in writing, formatting, and mathematical typesetting during the preparation of this report.

Chalakudy
May 2026

SUMMARY

About the Organisation:

Sacred Heart College (Autonomous), Chalakudy, is a government-aided institution affiliated with the University of Calicut. Established in 1980, the college currently offers ten undergraduate and nine postgraduate programmes, alongside a five-year integrated Psychology programme and three research specialisations. Guided by the motto “LUX IN TENEBRIS LUCET” (Light Shines in Darkness), the college is home to the Postgraduate and Research Department of Mathematics, an approved research centre under the University of Calicut.

Learning Outcomes:

- Understood the foundations of mathematical modelling and the classification of physical systems using differential equations.
- Derived the **1D transient heat equation** from conservation laws and Fourier’s Law, including the extension to temperature-dependent thermal diffusivity $\alpha(T)$.
- Studied and applied classical **boundary condition types** — Dirichlet, Neumann, and Robin — to physical problems.
- Obtained **analytical solutions** of the heat equation using the Fourier Series method implemented in Python(*Numpy*).
- Implemented **numerical simulations** using the Finite Difference Method (FDM) in Python for constant α .
- Compared analytical and numerical solutions through statistical validation using the *R programming language* to evaluate the accuracy, consistency, and reliability of the computational model.
- Learned to prepare scientific reports and typeset mathematical content using L^AT_EX.

Keywords: Mathematical Modeling, Partial Differential Equation, Heat Equation, Finite Difference method, Numerical Methods, Method Of Lines Fourier’s Law, Python, NumPY, Object Oriented Programming, Statistical Tools, R Programming

Contents

1	Introduction to Mathematical Modelling	7
1.1	Historical Background	7
1.2	Flow Of Model	8
1.3	The Heat Conduction Model	9
1.3.1	The Heat Equation	9
2	Basics and Classification	9
2.1	Deterministic vs Stochastic Models	10
2.2	Linear vs Non-linear Autonomous models	10
2.3	Statics vs Dynamic Models	11
2.4	Continuous vs Discrete Models	11
2.5	Autonomous vs Non-Autonomous	12
2.6	Category of Heat Conduction Model	12
3	Dimensional Analysis And Approximation	13
3.1	Linearization	13
4	Mathematical Formulation of Transient Heat Conduction	15
4.1	Derivation of the Conduction of Heat in a One-Dimensional Rod	15
4.1.1	Initial and Boundary Conditions	18
5	Analytical Solution	20
5.1	Step 1 — Steady State Solution	20
5.2	Step 2 — Homogeneous Boundary Conditions for v	21
5.3	Step 3 — Separation of Variables	21
5.4	Step 4 — Solving the ODEs	21
5.5	Step 5 — General Solution	22
5.6	Step 6 — Fourier Coefficients	22
5.7	Complete Analytical Solution	22
6	Analytical Data - Python Code	22
7	Numerical Solution — Finite Difference Method	25
7.1	The Finite Difference Grid	25
7.2	Crank - Nicolson Method	26
7.2.1	Gauss–Seidel Iterative Solver	27
8	Numerical Data - Python Code	27
9	Statistical Validation	32
9.1	Coefficient of determination R^2	32
9.2	Root Mean Square Error (RMSE)	33
9.3	Mean Absolute Percentage Error (MAPE)	33
9.4	Akaike Information Crite- rion (AIC)	34
10	Discussion and Conclusion	39

1 Introduction to Mathematical Modelling

Mathematics the language of universe! Not just poetry, but the eternal truth of every physical system that moves around the time. Mathematical modelling played a major role in shaping the scientific and technological advancements in human history.

Before delving into the description, let us first understand what a model is.

model (n): *a miniature representation of something; a pattern of something to be made; an example for imitation or emulation; a description or analogy used to help visualize something (e.g., an atom) that cannot be directly observed; a system of postulates, data and inferences presented as a mathematical description of an entity or state of affairs.* [6]

Then a **mathematical model (n):** *are representations in mathematical terms of the behaviour of real devices and objects.*

We want to learn how to make or generate such models, how to validate them, how to use them, and how and when their use is limited. But before moving deep — it is worth talking about why we do mathematical modelling.

*Mathematical modelling is the process of describing a **real-world** problem in mathematical terms, usually in the form of equations, and then using these equations both to help understand the original problem, and also to discover new features about the problem. Modelling both lies at the heart of much of our understanding of the world, and it allows engineers to design the technology of the future. With modelling we can travel to the edge of the universe, peer into the heart of the atom, and understand the future of our climate.* [5]

1.1 Historical Background

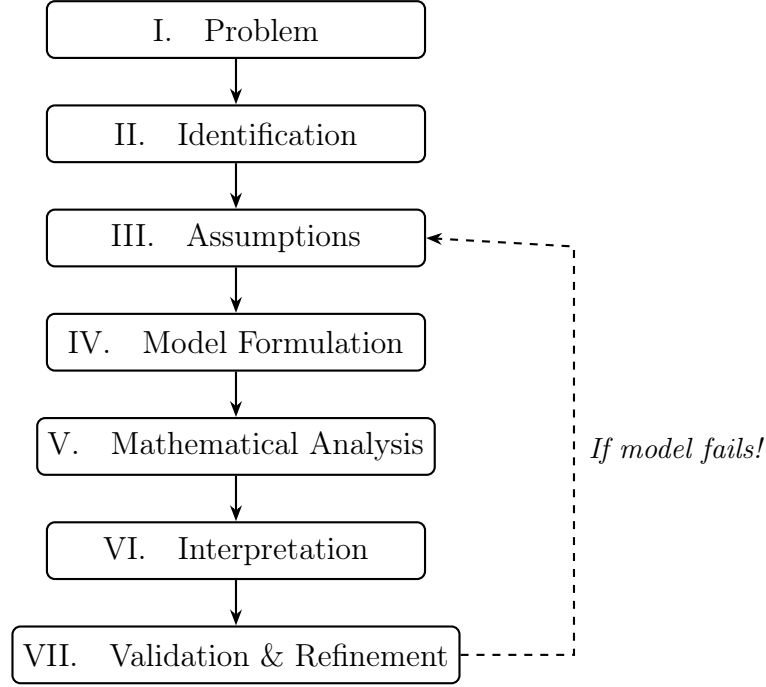
The roots of mathematical modelling begin from the ancient cultures where Babylonians, Egyptians, and Greeks used early mathematical techniques to solve practical problems such as land measurement, construction and architecture. Apart from those, scholars such as Al-Khwarizmi developed algebra, and innovations in trigonometry; Aryabhata introduced zero and decimal systems — all of which further reinforced the advancements in astronomy, optics, navigation and engineering. [21]

Further, the Renaissance and Enlightenment periods had remarkable advancements in mathematical modelling by the invention of calculus, enabling the modelling of dynamics and mechanical systems precisely in science and engineering. In the period of Industrial Revolution, mathematical modelling was further integrated into technological advancements, optimizing manufacturing processes and infrastructure design.

In the 20th century, the advent of computational power transformed mathematical modelling to the next level, allowing it to simulate complex systems and broaden its applications across different disciplines. As we move forward into the 21st century, the integration of mathematical modelling with data science, machine learning and artificial intelligence has the potential to address global challenges — ranging from climate change to healthcare advancements.

1.2 Flow Of Model

To build a mathematical model according to a problem we identified, does come up with certain steps [9]



So these are the steps, that we usually follow to model a problem. For a basic insight, let's consider population model as an example:

Population model or *Exponential growth model* are one of the major models in ecology. Let $N(t)$ be a population a time t . If each individual reproduces at rate r , then: [2]

$$\frac{dN}{dt} = r N, \underbrace{N(0) = N_0}_{\text{Initial Condition}} \quad (1)$$

Using the **variable separable method**:

$$\int \frac{dN}{N} = \int r dt \quad (2)$$

$$\ln(N) = rt + C \quad (3)$$

$$N = e^{rt+C} \quad (4)$$

$$N(t) = C e^{rt} \quad (5)$$

Applying initial conditions:

$$N(0) = N_0 \implies C = N_0 \quad (6)$$

Hence:

$$\boxed{N(t) = N_0 e^{rt}} \quad (7)$$

This is the way by which we model a system using mathematical tools, but throughout the article we mainly focus on **Heat Conduction Model**.

1.3 The Heat Conduction Model

The core element of every mathematical modelling is Differential Equations – equations that contain derivatives. Systems represented in these equations show change over certain parameters. [22] [17]

Here we mainly discuss the **Transient Heat Conduction in a 1D Rod**. This is one of the major models, which is written in partial differential equations (PDEs), and generally referred to as the **Heat Equation**. The heat equation describes how energy is transferred from one body to another as the result of a temperature gradient. It describes a wide variety of phenomena over a broad range of scales, from heat diffusion in nanoscale materials to the macroscopic motion of atmospheric air [3]. Apart from physical applications, its references lead to applications in other disciplines such as finance (Black-Scholes Model). Recently, its applications cover Biology (Tissue-Damage Prediction), Fractional and Complex modelling (modelling anomalous diffusion in composite or heterogeneous materials with complex geometry), Geophysics (Study of the subsurface temperature distribution, thermal skin depth), and Nanofluids.

1.3.1 The Heat Equation

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad (8)$$

[22]

where:

- $T(x, t)$ – temperature at position x and time t
- $\frac{\partial T}{\partial t}$ – rate of change of temperature with respect to time
- $\frac{\partial^2 T}{\partial x^2}$ – curvature of the temperature profile along the rod
- α – thermal diffusivity of the material $\left(\alpha = \frac{k}{\rho c_p} \right)$

Here, k is the thermal conductivity, ρ is the density, and c_p is the specific heat capacity of the material.

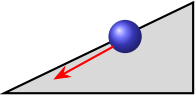
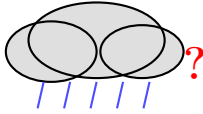
2 Basics and Classification

It is helpful to identify broad categories of models, so that classification of these models into these categories tells us immediately some of the essentials of their structure. One

division between models is based on the type of outcome they predict.

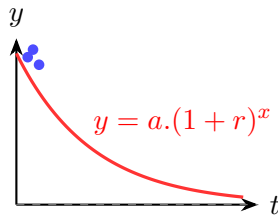
2.1 Deterministic vs Stochastic Models

Deterministic models ignore random variation, and so always predict the same outcome from a given starting point. On the other hand, the model may be more statistical in nature and so may predict the distribution of possible outcomes. Such models are said to be stochastic. [12] [14]

Examples :	
Deterministic  Ball rolling down a wedge	Stochastic  Tomorrow's weather
A ball rolling down a wedge follows a fixed, predictable path . Given the same initial conditions, the outcome is always the same.	Tomorrow's weather is uncertain and probabilistic . Even with the same conditions, outcomes can differ each time.

2.2 Linear vs Non-linear Autonomous models

According to the resulting equations, which may be algebraic, differential, or difference equations, being linear or non-linear, models are classified as linear or non-linear. A model is said to be linear when the variables are in the first power, which obeys superposition and can be solved analytically. In turn, non-linear models are those in which the variables have higher powers, which are more complex and can't be solved easily — they exhibit chaos! [12]

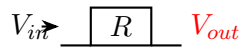
Examples :	
Linear  Car at constant speed	Non-Linear  Exponential decay of nuclear particle
Governed by $x = vt$, a first-power relationship. Obeys superposition — <i>double the time, double the distance</i> . Can be solved analytically.	Governed by $y = a.(1+r)^x$, a non-linear relationship in behaviour. The rate depends on the current quantity — <i>the more you have, the faster it decays</i> . Exhibits complex dynamics.

2.3 Statics vs Dynamic Models

In a static system, time doesn't play any part and hence the variables and the relationships describing the system are time independent. In contrast dynamic systems time plays an important role. [12]

Examples :

Static

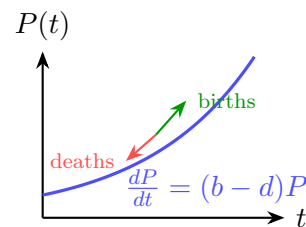


$$V_{out} = I \cdot R$$

Resistor Circuit

Output voltage V_{out} depends **strictly on the current input** — no memory of past states. The model captures a snapshot in time, with no evolution.

Dynamic



Population Model

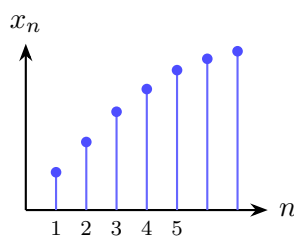
The future population **depends on current birth and death rates** — the model evolves over time. Past states directly influence future outcomes.

2.4 Continuous vs Discrete Models

A simple definition for a discrete and continuous models is that the dependant variable takes a certain discrete elements and gives discrete outputs which are characterized as discrete model, in contrast continuous models are based on continuous variables. [12]

Examples :

Discrete

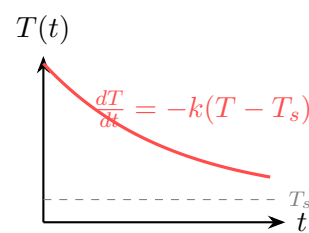


$$x_{n+1} = r x_n$$

Bank Account Balance

Defined at **distinct, separate points** in time — like monthly bank statements. The variable jumps from one step to the next with no values in between. Governed by *difference equations*.

Continuous



Newton's Law of Cooling

Defined **at every instant in time** — like temperature of a cooling object. The variable changes smoothly without any gaps. Governed by *differential equations*.

2.5 Autonomous vs Non-Autonomous

Autonomous systems operate independently based on their current state. Their governing equations do not explicitly depend on time, enabling self-regulation:

$$\frac{dx}{dt} = f(x)$$

Conversely, non-autonomous systems depend on external inputs or explicit time factors, requiring external control for behavioral changes:

$$\frac{dx}{dt} = f(x, t)$$

2.6 Category of Heat Conduction Model

In the earlier sections, we discussed the classification of mathematical models. Based on this understanding, let us now categorise the **heat conduction model**.

Classification of the Heat Equation	
Category	Heat Equation Properties
Deterministic	✓ Unique solution given same B.C & I.C
Linear	✓ Temperature appears only in first power
Dynamic	✓ Evolves over time t
Continuous	✓ Defined at every x and t
Autonomous	✓ No explicit t on right-hand side
Solvability	✓ Analytical solution exists under suitable conditions
Superposition	✓ Sum of solutions is also a solution

The one-dimensional heat equation is a deterministic, linear, continuous, and dynamic partial differential equation describing the diffusion of heat through a medium. Since the thermal diffusivity is treated as constant, the equation satisfies the principle of superposition and admits analytical solutions under appropriate boundary and initial conditions.

3 Dimensional Analysis And Approximation

Dimensional analysis is one of the major tools used in mathematics, physics, and engineering. It is a method by which the relationships between physical quantities are analyzed with respect to fundamental quantities and their dimensions, such as mass (M), length (L), time (T), temperature (θ or K), and electric current (A). It helps us understand how physical quantities are related and simplifies complex problems into simpler ones. [18]

Apart from that, dimensional analysis can be used to determine whether an equation is physically meaningful, derive relationships between variables, and convert units from one system to another (for example, from the SI system (kg, m, s) to the CGS system (g, cm, s)).

When we come to dimensional analysis, we must have knowledge of base quantities and derived quantities [23]. **Base quantities** are the fundamental physical quantities that are independent of one another and cannot be expressed in terms of other quantities. They serve as the foundation for measurement systems and dimensional analysis.

Derived quantities are the quantities obtained from base quantities through mathematical relationships such as multiplication, division, or powers. Their dimensions are expressed using combinations of fundamental dimensions. Examples include velocity, force, pressure, density, and energy. For example:

$$Velocity = \frac{Length}{Time} = [LT^{-2}]$$

One of the most important principles in dimensional analysis is **dimensional homogeneity** [23]. *Simply we could say that an apple can be added with another apple not an orange.* This principle ensures that an equation is physically consistent.

For example:

$$m = \frac{\vec{F}}{\vec{a}}$$

Let's check the dimensions

$$\text{Dimension of } \vec{F} = [MLT^{-2}]$$

$$\text{Dimension of } \vec{a} = [LT^{-2}]$$

$$\text{Dimension of } m = \frac{[MLT^{-2}]}{[LT^{-2}]} = [M] \text{ (Thus the equation is dimensionally homogenous.)}$$

3.1 Linearization

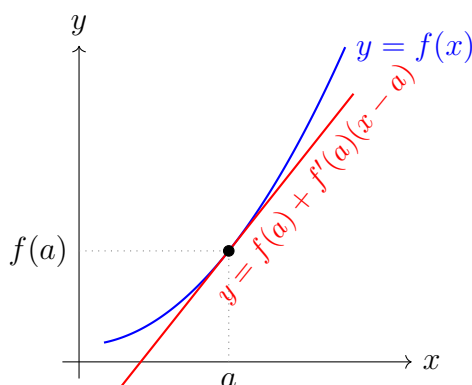
We often encounter difficulties while finding exact solutions for problems. If that is so, we often use **approximate** solutions for such cases, within some tolerance. Approximation is the process of finding a value which is close to the correct result. In mathematical modelling, approximation is necessary because: [20]

- Sometimes the data may be incomplete
- Presence of complex equations

- The exact values may not improve practical decisions

Here we are going to deal with linear approximation which is called **Linearization** by using the concept of tangent line.

Let f be a function differentiable at the point $x = a$. The equation of the tangent line to the graph of f at the point $x = a$ is $y = f(a) + f'(a)(x - a)$.



Definition [20]. The *linearization*, or *linear approximation*, of the function f near point $x = a$ is the linear function $L(x) = f(a) + f'(a)(x - a)$.

$$f(x) \approx L(x) \text{ near } x = a.$$

Example: Find the linear approximation to $f(x) = \sin x$ near $x = 0$

Solution. The linear approximation is the function $L(x) = f(a) + f'(a)(x - a)$, where $f(x) = \sin x$ and $a = 0$.

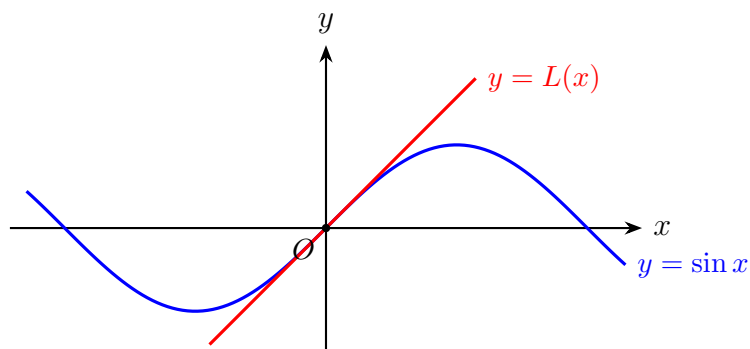
Since

$$f'(0) = f'(x) \Big|_{x=0} = (\sin x)' \Big|_{x=0} = \cos x \Big|_{x=0} = \cos 0 = 1$$

and $f(0) = \sin 0 = 0$, we find

$$L(x) = 0 + 1 \cdot (x - 0) \iff L(x) = x.$$

The linear function approximating $f(x) = \sin x$ near $x = 0$ is $L(x) = x$:



4 Mathematical Formulation of Transient Heat Conduction

As we discussed earlier, our main focus is on the heat conduction model. With reference to Section 1.2, before modeling a physical system, we need to identify the problem. Here, we are considering the heat flow through a 1D conductor; but before that, what is heat flow? [8] Heat energy is caused by the thermal agitation of molecular matter. There are basically two ways of transferring this energy: **Conduction** and **Convection**. *Conduction* results from collisions of neighboring molecules in which the kinetic energy of vibration of one molecule is transferred to its nearest neighbor. Thermal energy is thus spread by conduction even if the molecules themselves do not move their location appreciably. In addition, if a vibrating molecule moves from one region to another, it takes its thermal energy with it. This type of movement of thermal energy is called *Convection*. To begin simpler and as our main focus too, we consider conduction here.

4.1 Derivation of the Conduction of Heat in a One-Dimensional Rod

We begin by considering a rod of constant cross-sectional area A , oriented in the x -direction, where $x \in [0, L]$ [8, 1]. For the purpose of modeling we introduce amount of thermal energy per unit volume as a unknown variable and call it the **thermal energy density**:

$$e(x, t) \equiv \text{thermal energy density.}$$

We assume that all thermal quantities are constant across section; the rod is 1D. The simplest way is to insulate the lateral surface for avoiding the loss of heat through. The dependence on x and t corresponds to a situation in which the rod is not uniformly heated; the thermal energy density varies from one cross section to another.

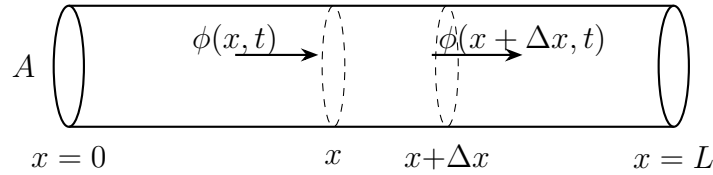


Figure 1: One-dimensional rod with heat energy flowing into and out of a thin slice.

Heat Energy, here we consider a thin slice of the rod $[x, x + \Delta x]$ as illustrated in Figure 1. If the thermal energy density is constant throughout the volume then total energy (In general, energy density isn't constant but as Δx is extremely small):

$$\text{heat energy.} = e(x, t)A\Delta x$$

Box 1 : Law of conservation of energy

According to the **Law of Conservation of energy** [1] the heat energy between $[x, x + \Delta x]$ changes in time due only to heat energy flowing across the edges and heat energy that generated inside (due to positive or negative sources of heat energy). Then;

rate of change of heat energy in time	=	heat energy flowing across boundaries per unit time	+	heat energy gen- erated inside per unit time.
---	---	---	---	--

Box 2 : Law of conservation of energy

This is called **conservation of heat energy**. For the small size, the rate of change of heat energy is

$$\frac{\partial}{\partial t}[e(x, t)A\Delta x], \quad (9)$$

where the partial derivative $\frac{\partial}{\partial t}$ is used because x is being held fixed.

Next we determine another parameter, **Heat flux** (ϕ):

$\phi(x, t)$ = the amount of thermal energy per unit time flowing to the right per unit surface area

The heat energy flowing per unit time across the boundaries of the slice is $\phi(x, t)A - \phi(x + \Delta x, t)A$. The sign of $\phi(x, t)$ determines the direction of heat flow:

- If $\phi(x, t) < 0$ — heat flows to the **left**
- If $\phi(x, t) > 0$ — heat flows to the **right**

As illustrated in Figure 1, when $\phi(x, t) > 0$ and $\phi(x + \Delta x, t) > 0$, the heat flow at x increases the energy in the slice, whereas the heat flow at $x + \Delta x$ decreases it.

heat sources. we also allow for internal sources of thermal energy:

$Q(x, t)$ = heat energy per unit volume generated per unit time,

perhaps due to chemical reactions or electrical heating. $Q(x, t)$ is approximately constant in space for a thin slice, and thus the total thermal energy generated per unit time in the thin slice is approximately $Q(x, t)A\Delta x$.

Substituting the parameters in *Box 2*; rate of change of heat energy is due to thermal energy flowing across the boundaries and internal sources:

$$\frac{\partial}{\partial t}[e(x, t)A\Delta x] \approx \phi(x, t)A - \phi(x + \Delta x, t)A + Q(x, t)A\Delta x. \quad (10)$$

Equation 10 assumes various quantities to be approximately constant over the small slice, and becomes increasingly accurate as $\Delta x \rightarrow 0$. Directly taking the limit $\Delta x \rightarrow 0$ gives $0 = 0$, which is uninformative. Instead, we first divide by Δx , and then take the limit $\Delta x \rightarrow 0$, to obtain:

$$\frac{\partial e}{\partial t} = \lim_{\Delta x \rightarrow 0} \frac{\phi(x, t) - \phi(x + \Delta x, t)}{\Delta x} + Q(x, t) \quad (11)$$

where the constant cross-sectional area has been canceled. We claim this result is exact (with no small errors), and hence we can replace $\approx$ by $=$ in *Equation 10*. In this limiting process, $\Delta x \rightarrow 0$, t is being fixed. Consequently, from the definition of PDE,

$$\frac{\partial e}{\partial t} = -\frac{\partial \phi}{\partial x} + Q \quad (12)$$

We usually describe materials by their **Temperature**,

$$T(x, t) = \text{Temperature}$$

not by their thermal energy density. And there is another important term we consider, the **Specific heat**

c = specific heat (the heat energy that must be supplied to a unit mass of a substance to raise its temperature one unit).

Note: *In general, the specific heat c of a material depends on temperature T . In the 1D rod case, c depends on position x , i.e., $c = c(x)$. However, assuming the rod is made of a single uniform material, we treat c as a constant throughout.*

Thermal energy. The thermal energy in a thin slice is $e(x, t)A \Delta x$, which is also the energy required to raise the temperature from a reference temperature 0 to $T(x, t)$. Since c is independent of temperature, the heat energy per unit mass is $c(x)T(x, t)$. We thus introduce the **mass density** $\rho(x)$:

$$\rho(x) = \text{mass density (mass per unit volume),}$$

as it's a uniform rod we take ρ instead; From *Box 1* we know total thermal energy in the slice, we equate that with $c(x)T(x, t) \cdot \rho A \Delta x$, so that

$$e(x, t)A \Delta x = cT(x, t) \cdot \rho A \Delta x$$

canceling $A \Delta x$;

$$e(x, t) = c\rho T(x, t) \quad (13)$$

Relating Eqn 13 with Eqn 12, then we get;

$$c\rho \frac{\partial T}{\partial t} = -\frac{\partial \phi}{\partial x} + Q \quad (14)$$

According to **Fourier's Law of Heat Conduction**. [24] The heat flux $\phi(x, t)$ is proportional to the temperature gradient, and heat flows in the direction of decreasing temperature [2]:

$$\phi(x, t) = -k_0 \frac{\partial T}{\partial x} \quad (15)$$

where:

- ϕ — heat flux; heat energy flowing per unit time per unit surface area
- k_0 — thermal conductivity of the material ($W/m \cdot K$)
- $\frac{\partial T}{\partial x}$ — temperature gradient
- The negative sign ensures heat flows from **hot** $\rightarrow$ **cold**

If Eqn 15 is substituted into Eqn 14, then

$$c\rho\frac{\partial T}{\partial t} = \frac{\partial}{\partial x}(K_0\frac{\partial T}{\partial x}) + Q$$

In addition, there are no sources, let $Q = 0$ and c, ρ, K_0 are constants, then;

$$c\rho\frac{\partial T}{\partial t} = K_0\frac{\partial^2 T}{\partial x^2}$$

And finally we get;

$$\underbrace{\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}}_{\text{The Heat Equation}} \quad (16)$$

The Heat Equation

where $\alpha = \frac{K_0}{c\rho}$ is the **thermal diffusivity** of the material — a constant that controls how fast heat spreads through the rod.

4.1.1 Initial and Boundary Conditions

To completely define the heat conduction problem, the PDE alone is not sufficient. We need additional information about:

- The **initial state** of the rod at $t = 0$
- The **behaviour at the boundaries** $x = 0$ and $x = L$ for all time $t > 0$

These are called **Initial Conditions (IC)** and **Boundary Conditions (BC)** respectively. Together with the PDE, they form a **well-posed problem** — meaning a unique solution exists. [20]

Initial Condition. The initial condition specifies the temperature distribution throughout the rod at time $t = 0$:

$$T(x, 0) = f(x), \quad 0 \leq x \leq L \quad (17)$$

In the simplest case, the rod starts at a **uniform temperature** T_0 :

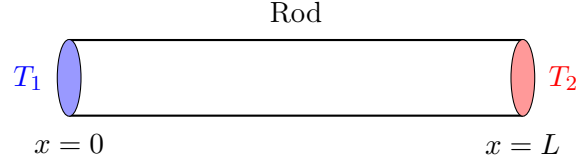
$$T(x, 0) = T_0$$

Boundary Conditions. The boundary conditions describe what happens at the two ends of the rod ($x = 0$ and $x = L$) for all $t > 0$. There are three classical types: [20]

1. Dirichlet Boundary Condition (First Kind)

The temperature is **prescribed** at the boundary. This is the most physically intuitive condition — we simply fix the temperature at the ends of the rod, like holding one end in ice and the other in a flame.

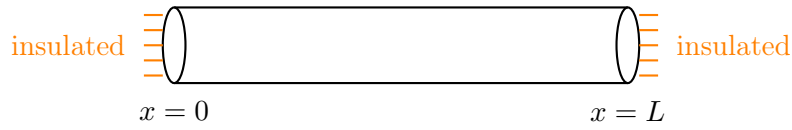
$$T(0, t) = T_1, \quad T(L, t) = T_2, \quad t > 0 \quad (18)$$



2. Neumann Boundary Condition (Second Kind)

The **heat flux** is prescribed at the boundary, meaning we control how much heat flows in or out. The special case of **zero flux** means the end is perfectly *insulated* — no heat escapes.

$$\left. \frac{\partial T}{\partial x} \right|_{x=0} = 0, \quad \left. \frac{\partial T}{\partial x} \right|_{x=L} = 0, \quad t > 0 \quad (19)$$



3. Robin Boundary Condition (Third Kind)

A **combination** of Dirichlet and Neumann — the heat flux at the boundary is proportional to the difference between the surface temperature and the surrounding (ambient) temperature T_∞ . This is physically described by *Newton's Law of Cooling*:

$$-k \left. \frac{\partial T}{\partial x} \right|_{x=L} = h(T - T_\infty), \quad t > 0 \quad (20)$$

where h is the **heat transfer coefficient** ($W/m^2 \cdot K$) and T_∞ is the ambient temperature. The rod naturally loses more heat when it is hotter than its surroundings.

Summary of Boundary Conditions		
Type	Condition	Physical Meaning
Dirichlet	$T = T_1$ at boundary	Fixed temperature
Neumann	$\frac{\partial T}{\partial x} = 0$ at boundary	Insulated / fixed flux
Robin	$-k \frac{\partial T}{\partial x} = h(T - T_\infty)$	Convective heat loss

In this project, we adopt **Dirichlet boundary conditions** with a uniform initial temperature, forming the complete problem statement:

$\frac{\partial T}{\partial t} = \frac{\partial^2 T}{\partial x^2}, \quad 0 < x < L, \quad t > 0$
$T(x, 0) = T_0 \quad (\text{Initial Condition})$
$T(0, t) = T_1, \quad T(L, t) = T_2 \quad (\text{Boundary Conditions})$

This is the complete mathematical model that will be solved numerically in the following sections.

5 Analytical Solution

In mathematical modeling, the equations are often a set of differential equations [4]. To obtain meaningful results, we solve them analytically. Here we develop the solution of the 1D heat equation for an **iron rod** where one end is heated (candle) while the other is cold, giving boundary conditions $T(0, t) = 100^\circ\text{C}$ and $T(L, t) = 0^\circ\text{C}$.

We solve using the **Fourier series method**, but a problem arises — the Fourier sine series always gives **zero** at both boundaries:

$$\sin\left(\frac{n\pi \cdot 0}{L}\right) = 0, \quad \sin\left(\frac{n\pi \cdot L}{L}\right) = 0$$

but our boundary condition at $x = 0$ is 100°C , not zero. To resolve this, we introduce a steady state $T_{steady}(x)$ and decompose [8]:

$$T(x, t) = T_{steady}(x) + \underbrace{v(x, t)}_{\text{Transient Part}} \quad (21)$$

where $v(x, t) = T(x, t) - T_{steady}(x)$ now carries only **zero** boundary conditions, which Fourier series handles naturally.

5.1 Step 1 — Steady State Solution

The steady state is the temperature distribution as $t \rightarrow \infty$, when the rod stops changing:

$$\frac{\partial T}{\partial t} = 0 \implies \frac{d^2 T_{steady}}{dx^2} = 0 \quad (22)$$

General solution is a straight line $T_{steady} = ax + b$. Applying boundary conditions:

$T_{steady}(x) = 100 - \frac{100x}{L}$
--

(23)

A ramp from 100°C to 0°C — the final destination of the rod as $t \rightarrow \infty$.

5.2 Step 2 — Homogeneous Boundary Conditions for v

Substituting $v = T - T_{steady}$ into the boundary conditions [8]:

$$v(0, t) = 100 - 100 = 0 \quad (24)$$

$$v(L, t) = 0 - 0 = 0 \quad (25)$$

Both boundaries are zero. Since T_{steady} is linear, $\frac{d^2 T_{steady}}{dx^2} = 0$, so v satisfies the same PDE:

$$\frac{\partial v}{\partial t} = \alpha \frac{\partial^2 v}{\partial x^2} \quad (26)$$

with modified initial condition:

$$v(x, 0) = 25 - \left(100 - \frac{100x}{L}\right) = -75 + \frac{100x}{L} \quad (27)$$

5.3 Step 3 — Separation of Variables

Assuming $v(x, t) = X(x) \cdot \mathcal{T}(t)$ [13, 22] and substituting into the PDE:

$$\frac{\mathcal{T}'(t)}{\alpha \mathcal{T}(t)} = \frac{X''(x)}{X(x)} = -\lambda \quad (28)$$

Since left side depends only on t and right side only on x , both equal the same constant $-\lambda$, giving two ODEs:

$$X''(x) + \lambda X(x) = 0 \quad (29)$$

$$\mathcal{T}'(t) + \alpha \lambda \mathcal{T}(t) = 0 \quad (30)$$

5.4 Step 4 — Solving the ODEs

Time ODE — Separating variables and integrating [22]:

$$\frac{d\mathcal{T}}{\mathcal{T}} = -\alpha \lambda dt \implies \boxed{\mathcal{T}(t) = C e^{-\alpha \lambda t}} \quad (31)$$

Space ODE — Characteristic equation $r^2 + \lambda = 0$ gives $r = \pm i\sqrt{\lambda}$. Using Euler's formula $e^{i\theta} = \cos \theta + i \sin \theta$:

$$X(x) = A \sin(\sqrt{\lambda} x) + B \cos(\sqrt{\lambda} x) \quad (32)$$

Applying $X(0) = 0$ gives $B = 0$. Applying $X(L) = 0$:

$$\sin(\sqrt{\lambda_n}L) = 0 \implies \boxed{\lambda_n = \left(\frac{n\pi}{L}\right)^2, \quad n = 1, 2, 3, \dots} \quad (33)$$

These are the **eigenvalues**, with eigenfunctions $X_n(x) = \sin\left(\frac{n\pi x}{L}\right)$.

5.5 Step 5 — General Solution

By the **superposition principle** [13]:

$$v(x, t) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right) e^{-\alpha\left(\frac{n\pi}{L}\right)^2 t} \quad (34)$$

5.6 Step 6 — Fourier Coefficients

At $t = 0$ the exponential equals 1. Using **orthogonality** of sine functions:

$$\int_0^L \sin\left(\frac{n\pi x}{L}\right) \sin\left(\frac{m\pi x}{L}\right) dx = \begin{cases} 0 & n \neq m \\ \frac{L}{2} & n = m \end{cases} \quad (35)$$

Multiplying both sides by $\sin\left(\frac{m\pi x}{L}\right)$ and integrating, all terms vanish except $n = m$:

$$\boxed{B_n = \frac{2}{L} \int_0^L v(x, 0) \sin\left(\frac{n\pi x}{L}\right) dx} \quad (36)$$

Evaluating using integration by parts:

$$B_n = \frac{2}{n\pi} [100(-1)^{n+1} - 75(1 - (-1)^n)] \quad (37)$$

5.7 Complete Analytical Solution

The complete analytical solution [13]:

$$\boxed{T(x, t) = \underbrace{\left(100 - \frac{100x}{L}\right)}_{T_{steady}(x)} + \underbrace{\sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right) e^{-\alpha\left(\frac{n\pi}{L}\right)^2 t}}_{v(x, t)} \quad (38)$$

6 Analytical Data - Python Code

It is a herculean task to obtain a large number of data points using pen and paper, which leads to significant time complexity. This is where computational power becomes essential. By using the Python programming language, we can code the problem to obtain both raw numerical data and visual representations efficiently.

```

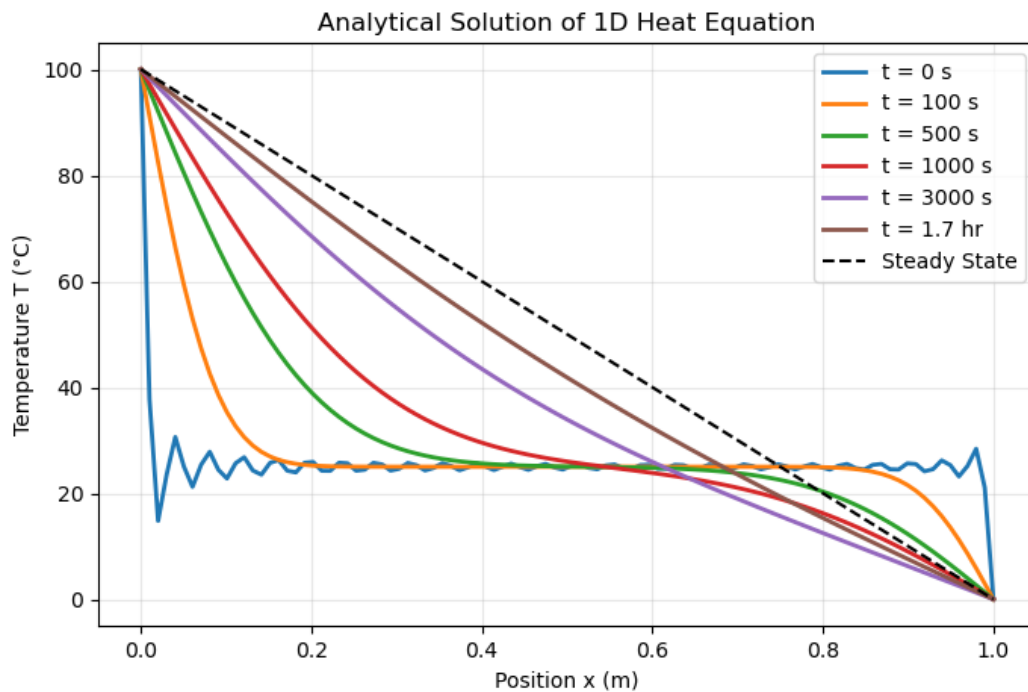
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class HeatEquation1D:
5     def __init__(self, L, alpha, T_left, T_right,
6                 T_initial, N=50, nx=100):
7         self.L = L
8         self.alpha = alpha
9         self.T_left = T_left
10        self.T_right = T_right
11        self.T_initial = T_initial
12        self.N = N
13        self.x = np.linspace(0, L, nx)
14        # Steady state solution
15        self.T_steady = (
16            T_left + (T_right - T_left) * self.x / L
17        )
18        # Fourier coefficient
19        def _Bn(self, n):
20            integrand = (
21                (self.T_initial - self.T_steady)
22                * np.sin(n * np.pi * self.x / self.L)
23            )
24            return (
25                2 / self.L
26            ) * np.trapezoid(integrand, self.x)
27        # Analytical solution
28        def solve(self, t):
29            T = self.T_steady.copy()
30            for n in range(1, self.N + 1):
31                decay = np.exp(
32                    -self.alpha
33                    * (n * np.pi / self.L) ** 2
34                    * t
35                )
36                T += (
37                    self._Bn(n)
38                    * decay
39                    * np.sin(n * np.pi * self.x / self.L)
40                )
41            return T
42        # Plot solution
43        def plot(self, t_list):
44            plt.figure(figsize=(8, 5))
45            for t in t_list:
46                T = self.solve(t)
47                label = (
48                    f't = {t} s'
49                    if t < 3600
50                    else f't = {t/3600:.1f} hr'
51                )
52                plt.plot(self.x, T, linewidth=2, label=label)
53            # Steady state
54            plt.plot(
55                self.x,
56                self.T_steady,
57                'k--',

```

```

58         label='Steady State'
59     )
60     plt.xlabel('Position x (m)')
61     plt.ylabel('Temperature T ( C )')
62     plt.title(
63         'Analytical Solution of 1D Heat Equation'
64     )
65     plt.grid(True, alpha=0.3)
66     plt.legend()
67     plt.show()
68 # =====
69 # MAIN
70 # =====
71 if __name__ == "__main__":
72     rod = HeatEquation1D(
73         L=1.0,
74         alpha=2.3e-5,
75         T_left=100.0,
76         T_right=0.0,
77         T_initial=25.0,
78         N=50,
79         nx=100
80     )
81     t_list = [0, 100, 500, 1000, 3000, 6000]
82     rod.plot(t_list)

```



Plot obtained by Analytical solution

7 Numerical Solution — Finite Difference Method

The Finite Difference Method (FDM) approximates derivatives by replacing them with **difference quotients** at discrete grid points. The continuous rod is divided into N equally spaced points with spacing $\Delta x = h$, and time is advanced in steps $\Delta t = k$.

7.1 The Finite Difference Grid

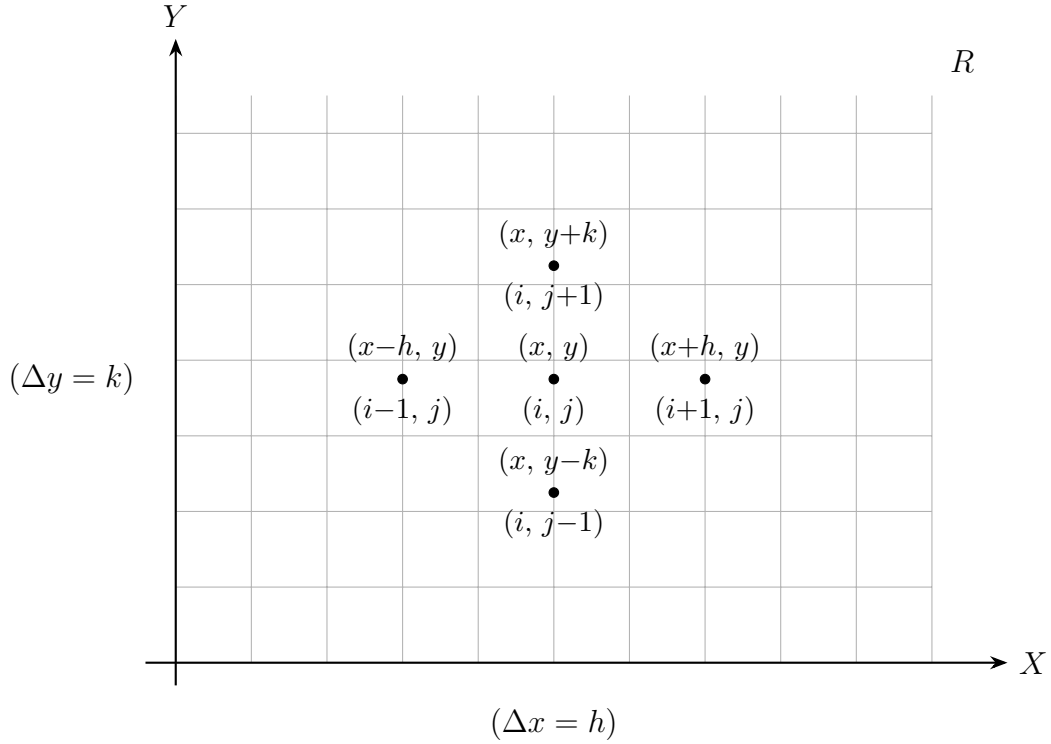


Figure 2: Finite Difference Grid — stencil points around (i, j)

Then we have the finite difference approximations for the partial derivatives in the x -direction:

$$\begin{aligned} \frac{\partial u}{\partial x} &= \frac{u(x+h, y) - u(x, y)}{h} + O(h) = \frac{u(x, y) - u(x-h, y)}{h} + O(h) \\ &= \frac{u(x+h, y) - u(x-h, y)}{2h} + O(h^2) \\ \frac{\partial^2 u}{\partial x^2} &= \frac{u(x-h, y) - 2u(x, y) + u(x+h, y)}{h^2} + O(h^2) \end{aligned} \quad (39)$$

Writing $u(ih, jk)$ as simply $u_{i,j}$, the above approximations become:

$$u_x = \frac{u_{i+1,j} - u_{i,j}}{h} + O(h) \quad (40)$$

$$= \frac{u_{i,j} - u_{i-1,j}}{h} + O(h) \quad (41)$$

$$= \frac{u_{i+1,j} - u_{i-1,j}}{2h} + O(h^2) \quad (42)$$

$$u_{xx} = \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h^2} + O(h^2) \quad (43)$$

Similarly we have the approximations for the derivatives w.r.t. y :

$$u_y = \frac{u_{i,j+1} - u_{i,j}}{k} + O(k) \quad (44)$$

$$= \frac{u_{i,j} - u_{i,j-1}}{k} + O(k) \quad (45)$$

$$= \frac{u_{i,j+1} - u_{i,j-1}}{2k} + O(k^2) \quad (46)$$

$$u_{yy} = \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{k^2} + O(k^2) \quad (47)$$

Replacing the derivatives in any partial differential equation by their corresponding difference approximations (40)–(47), we obtain the **finite-difference analogues** of the given equation.

7.2 Crank - Nicolson Method

This particular method is employed to solve the heat equation numerically. Unlike the explicit method, the **Crank–Nicolson method** is an implicit finite difference scheme, which provides improved stability and reduced numerical error, thereby yielding more accurate solutions.

Here the $\frac{\partial^2 T}{\partial x^2}$ is replaced by the average of its central- difference approximations on the j th and $(j+1)$ th time rows. Thus the eqn is reduced to [7]

$$\frac{T_{i,j+1} - T_{i,j}}{k} = \frac{\alpha}{2} \left[\frac{T_{i-1,j} - 2T_{i,j} + T_{i+1,j}}{h^2} + \frac{T_{i-1,j+1} - 2T_{i,j+1} + T_{i+1,j+1}}{h^2} \right] \quad (48)$$

Let the mesh ratio $r = \frac{\alpha k}{h^2}$. Multiplying both sides by $2k$ and rearranging:

$$-r T_{i-1,j+1} + (2 + 2r) T_{i,j+1} - r T_{i+1,j+1} = r T_{i-1,j} + (2 - 2r) T_{i,j} + r T_{i+1,j} \quad (49)$$

This can be written compactly as [7]:

$$\boxed{-r T_{i-1}^{n+1} + (2 + 2r) T_i^{n+1} - r T_{i+1}^{n+1} = r T_{i-1}^n + (2 - 2r) T_i^n + r T_{i+1}^n} \quad (50)$$

where superscript n denotes the time level and subscript i the spatial node. After the discretization of the PDE using CNM, we obtain an algebraic discretized system.

7.2.1 Gauss–Seidel Iterative Solver

The Crank–Nicolson discretization leads to a tridiagonal system of algebraic equations at each time step. Instead of solving the resulting matrix equation directly, an iterative technique such as the **Gauss–Seidel method** can be employed. [7]

Rewriting Eq. (52) in iterative form:

$$-rT_{i-1}^{n+1} + (2 + 2r)T_i^{n+1} - rT_{i+1}^{n+1} = rT_{i-1}^n + (2 - 2r)T_i^n + rT_{i+1}^n$$

Solving for T_i^{n+1} ,

$$T_i^{n+1} = \frac{rT_{i-1}^{n+1} + rT_{i+1}^{n+1} + rT_{i-1}^n + (2 - 2r)T_i^n + rT_{i+1}^n}{2 + 2r}$$

In the Gauss–Seidel iterative scheme, the most recently updated values are immediately used during the iteration process. Hence, the iterative form becomes

$$T_i^{(k+1)} = \frac{rT_{i-1}^{(k+1)} + rT_{i+1}^{(k)} + rT_{i-1}^n + (2 - 2r)T_i^n + rT_{i+1}^n}{2 + 2r}$$

where k denotes the iteration number.

The iterations are repeated until the convergence criterion is satisfied:

$$\left| T_i^{(k+1)} - T_i^{(k)} \right| < \epsilon$$

where ϵ is the prescribed tolerance.

The Gauss–Seidel method is computationally efficient for solving the tridiagonal systems arising from the Crank–Nicolson discretization of the one-dimensional heat equation. Compared to direct matrix inversion techniques, the iterative approach reduces computational complexity and is well suited for numerical implementation using Python.

8 Numerical Data - Python Code

In the previous sections, we discussed the numerical methods used to solve partial differential equations through discretization using the Crank–Nicolson method and iterative solving using the Gauss–Seidel method to reduce computational complexity. In this section, we implement the developed numerical scheme using Python programming [19].

Source code:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5
6 class HeatRod:
7     def __init__(self, length, num_points, diffusivity,
8                 time_step, num_steps):
9
10        self.length      = length
11        self.num_points   = num_points
12        self.diffusivity = diffusivity    # fixed typo: was 'diffisivity'
13
14        self.time_step    = time_step
15        self.num_steps    = num_steps
16
17        self.spacing = length / (num_points - 1)
18        self.r        = diffusivity * time_step / self.spacing**2
19
20        self.positions = np.linspace(0, length, num_points)
21        self.temperature = np.full(num_points, 25.0)
22
23        self.temperature[0] = 100.0
24        self.temperature[-1] = 0.0
25
26        self.saved_profiles = {}
27        self.saved_times    = []
28
29        print("Rod is ready!")
30        print(f"   Length           : {self.length} m")
31        print(f"   Grid points        : {self.num_points}")
32        print(f"   Grid spacing (dx)  : {self.spacing:.4f} m")
33        print(f"   Time step (dt)     : {self.time_step} s")
34        print(f"   Total time         : {self.num_steps * self.time_step} s")
35        print(f"   Mesh ratio r       : {self.r:.4f}")
36
37        # -----
38        # STEP 1: Build the right-hand side
39        # -----
40
41        def build_right_side(self):
42            rhs = np.zeros(self.num_points)
43            for i in range(1, self.num_points - 1):
44                rhs[i] = (
45                    self.r * self.temperature[i - 1]
46                    + (2 - 2 * self.r) * self.temperature[i]
47                    + self.r * self.temperature[i + 1]
48                )
49            return rhs
50
51        # -----
52        # STEP 2: Solve for next temperature using Gauss-Seidel
53        # -----
54
55        def solve_next_step(self, rhs):
```

```

55     new_temperature = self.temperature.copy()
56     tolerance       = 1e-6
57     max_iterations   = 1000
58
59     for iteration in range(max_iterations):
60         old_temperature = new_temperature.copy()
61
62         for i in range(1, self.num_points - 1):
63             new_temperature[i] = (
64                 rhs[i]
65                 + self.r * new_temperature[i - 1]
66                 + self.r * old_temperature[i + 1]
67             ) / (2 + 2 * self.r)
68
69         new_temperature[0] = 100.0
70         new_temperature[-1] = 0.0
71
72         change = np.max(np.abs(new_temperature - old_temperature))
73         if change < tolerance:
74             break
75
76     return new_temperature
77
78     # -----
79     # STEP 3: Run simulation, saving at specific time points
80     # -----
81
82     def run(self, save_at=None):
83         """
84         save_at : list of exact time values (in seconds) to snapshot.
85                   Must be multiples of time_step.
86                   Defaults to match the analytical t_list.
87         """
88         if save_at is None:
89             save_at = [0, 100, 500, 1000, 3000, 6000]
90
91         save_at_set = set(save_at)
92
93         # Save t = 0
94         self.saved_profiles[0] = self.temperature.copy()
95         self.saved_times.append(0)
96
97         print("\nSimulating heat flow through the rod...")
98
99         for step in range(self.num_steps):
100
101             rhs = self.build_right_side()
102             self.temperature = self.solve_next_step(rhs)
103
104             current_time = (step + 1) * self.time_step
105
106             if current_time in save_at_set:
107                 self.saved_profiles[current_time] = self.temperature.
108                     copy()
109                 self.saved_times.append(current_time)
110
111         # Summary table
112         mid = self.num_points // 2

```

```

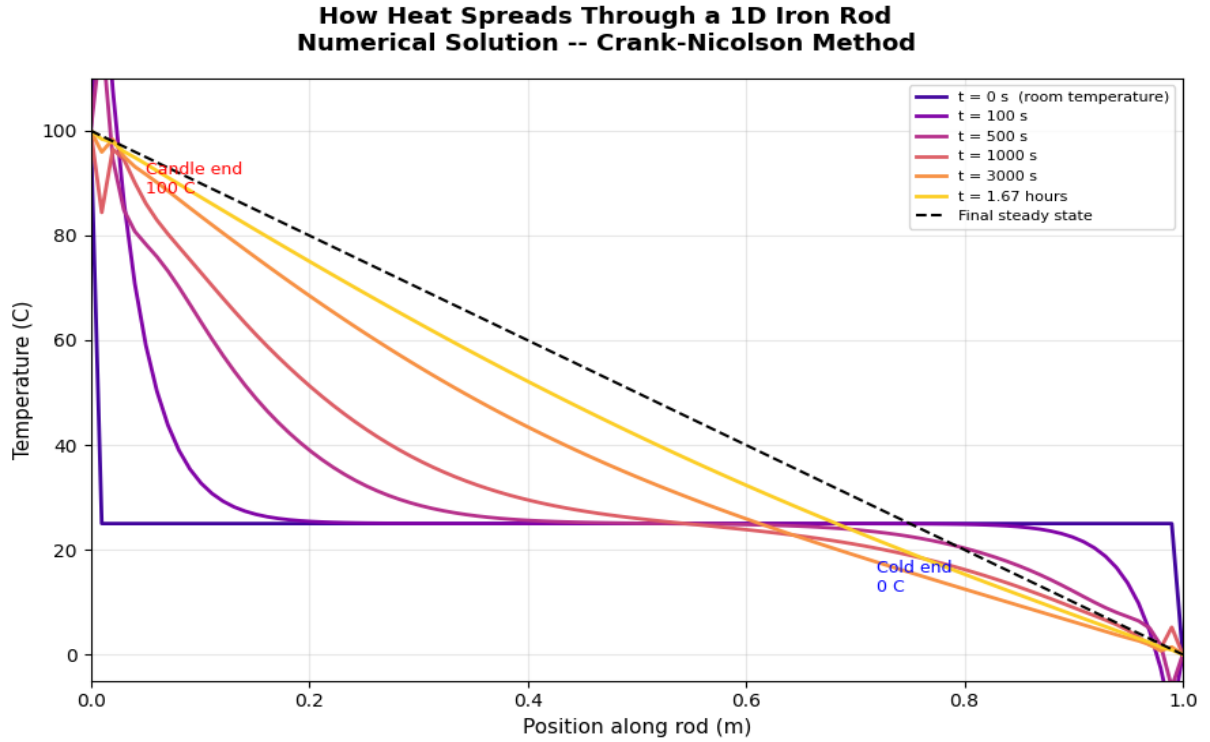
112     print(f"\n{'':->50}")
113     print(f"{'Time (s)':<12} {'Midpoint Temp (C)':<20} {'Status'}")
114     print(f"{'':->50}")
115     for t, T in sorted(self.saved_profiles.items()):
116         status = "initial" if t == 0 else \
117                 "final" if t == max(self.saved_profiles) else ""
118         print(f"{'t':<12.0f} {'T[mid]:<20.4f} {'status'}")
119     print(f"{'':->50}")
120     print("Simulation complete!")
121
122     # -----
123     # STEP 4: Plot
124     # -----
125
126     def plot(self, save_as='fdm_heat_solution.png'):
127         fig, ax = plt.subplots(figsize=(9, 6))
128         fig.suptitle(
129             'How Heat Spreads Through a 1D Iron Rod\n'
130             'Numerical Solution -- Crank-Nicolson Method',
131             fontsize=13, fontweight='bold'
132         )
133
134         profiles = sorted(self.saved_profiles.items())
135         colors = plt.cm.plasma(np.linspace(0.1, 0.9, len(profiles)))
136
137         for i, (t, T) in enumerate(profiles):
138             if t == 0:
139                 label = 't = 0 s (room temperature)'
140             elif t >= 3600:
141                 label = f't = {t/3600:.2f} hours'
142             else:
143                 label = f't = {t:.0f} s'
144
145             ax.plot(self.positions, T,
146                     color=colors[i], linewidth=2, label=label)
147
148             steady = 100.0 - 100.0 * self.positions / self.length
149             ax.plot(self.positions, steady, 'k--',
150                     linewidth=1.5, label='Final steady state')
151
152             ax.set_xlabel('Position along rod (m)', fontsize=11)
153             ax.set_ylabel('Temperature (C)', fontsize=11)
154             ax.legend(fontsize=8, loc='upper right')
155             ax.grid(True, alpha=0.3)
156             ax.set_xlim([0, self.length])
157             ax.set_ylim([-5, 110])
158
159             ax.annotate('Candle end\n100 C',
160                         xy=(0, 100), xytext=(0.05, 88),
161                         fontsize=9, color='red')
162             ax.annotate('Cold end\n0 C',
163                         xy=(1, 0), xytext=(0.72, 12),
164                         fontsize=9, color='blue')
165
166         plt.tight_layout()
167         plt.savefig(save_as, dpi=150, bbox_inches='tight')
168         print(f"\nPlot saved as {save_as}")
169

```

```

170 # -----
171 # STEP 5: Save CSV
172 # -----
173
174 def save_csv(self, filename='fdm_heat_data.csv'):
175     rows = []
176     for t, T in sorted(self.saved_profiles.items()):
177         for position, temp in zip(self.positions, T):
178             rows.append({
179                 'x (m)' : round(float(position), 4),
180                 't (s)' : round(float(t), 2),
181                 'T_numerical (C)' : round(float(temp), 6)
182             })
183
184     data = pd.DataFrame(rows)
185     data.to_csv(filename, index=False)
186
187     print(f"Data saved to {filename}")
188     print(f"Total rows : {len(data)}")
189     print(f"\nSample:")
190     print(data.head(6).to_string(index=False))
191
192     return data
193
194
195 # =====
196 # RUN THE SIMULATION
197 # =====
198
199 if __name__ == "__main__":
200
201     # t_list must match the analytical solution exactly
202     t_list = [0, 100, 500, 1000, 3000, 6000] # 7 snapshots
203
204     rod = HeatRod(
205         length = 1.0,
206         num_points = 100, # matches analytical nx=100 100 x-
207             points
208         diffusivity = 2.3e-5,
209         time_step = 100, # dt=100 s (all t_list values are
210             multiples)
211         num_steps = 60 # covers up to t = 6000 s
212     )
213
214     rod.run(save_at=t_list) # 7 snapshots 100 points =
215     700 rows
216
217     rod.plot(save_as='fdm_heat_solution.png')
218
219     rod.save_csv(filename='fdm_heat_data.csv')
220
221     print("\nAll done!")

```



Plot obtained by Numerical solution

We had obtained the numerical data points for the heat equation under the assumption of constant thermal diffusivity. Now we will compare both analytical and numerical data sets by statistical methods.

9 Statistical Validation

Statistical validation demonstrates strong agreement between the numerical and analytical solutions. To evaluate the accuracy and reliability of the obtained results, statistical measures such as the coefficient of determination (R^2), Root Mean Square Error (RMSE), Mean Absolute Percentage Error (MAPE), and Akaike Information Criterion (AIC) are employed. These methods provide a clearer quantitative assessment of the agreement and predictive capability of the numerical model [11].

Before taking part in the operation of fitting let's go through each one:

9.1 Coefficient of determination R^2

R-squared is a measure used in statistics to quantify how well a model demonstrates the variation of a dependent variable through its independent variables. It ranges from 0 to 1, where 1 indicates a perfect fit of the model to the data. [11]

Coefficient of Determination (R^2)

$$R^2 = \frac{SSR}{SST}$$

$$SSR = \sum (\hat{y}_i - \bar{y})^2$$

$$SST = \sum (y_i - \bar{y})^2$$

Where:

- R^2 : Coefficient of determination measuring the goodness-of-fit.
- SSR : Sum of Squares due to Regression, representing the variation explained by the model.
- SST : Total Sum of Squares, representing the total variation present in the observed data.
- $\hat{y}_i$: Predicted or numerical solution values.
- y_i : Analytical or observed values.
- $\bar{y}$: Mean of the observed values.

9.2 Root Mean Square Error (RMSE)

The root mean square error (RMSE) measures the average difference between a statistical model's predicted values and the actual values. Mathematically, it is the standard deviation of the residuals. [10]

Root Mean Square Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Where:

- $RMSE$: Root Mean Square Error measuring the average deviation between analytical and numerical solutions.
- n : Total number of data points.
- y_i : Analytical or observed values.
- $\hat{y}_i$: Predicted or numerical solution values.
- $(y_i - \hat{y}_i)$: Residual error between the analytical and numerical solutions.

9.3 Mean Absolute Percentage Error (MAPE)

Mean Absolute Percentage Error (MAPE): What You Need To Know ...The Mean Absolute Percentage Error (MAPE) is a statistical metric used to measure the accuracy of forecasting models. It calculates the average absolute percentage difference between predicted and actual values, expressing accuracy as a clear, intuitive percentage. [16]

Mean Absolute Percentage Error (MAPE)

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Where:

- $MAPE$: Mean Absolute Percentage Error representing the average percentage deviation between analytical and numerical solutions.
- n : Total number of data points.
- y_i : Analytical or observed values.
- $\hat{y}_i$: Predicted or numerical solution values.
- $(y_i - \hat{y}_i)$: Absolute error between the analytical and numerical solutions.

9.4 Akaike Information Criterion (AIC)

The AIC is a model selection tool i.e. a method of comparing two or more candidate regression models. The AIC methodology attempts to find the model that best explains the data with a minimum of parameters [15].

Akaike Information Criterion (AIC)

$$AIC = 2k - 2 \ln(L)$$

Where:

- AIC : Akaike Information Criterion used for measuring the quality of a statistical model.
- k : Number of estimated parameters in the model.
- L : Maximum likelihood function of the model.
- $\ln(L)$: Natural logarithm of the likelihood function.

A lower value of AIC indicates a better balance between model accuracy and complexity.

Now we have a basic understanding of these statistical validation tools. By considering the analytical solution as the true or reference value, the numerical model is fitted and evaluated against it. The statistical measures provide a quantitative assessment of the agreement between the analytical and numerical solutions, thereby helping to determine the accuracy, reliability, and predictive capability of the numerical method. For the easiness of validation and direct tools and libraries we consider *R Programming language* for this purpose. [4]

R Code - For Statistical validation

```
1 # =====
2 # STATISTICAL VALIDATION OF HEAT EQUATION SOLUTION
3 # =====
4
5 # -----
6 # 1. LOAD LIBRARIES
7 # -----
```

```

8
9  install.packages("readr")
10
11 library(readr)
12
13 # -----
14 # 2. READ CSV FILES
15 # -----
16
17 analytical <- read_csv("/content/analytical_solution.csv")
18 numerical  <- read_csv("/content/numerical_solution.csv")
19
20 # -----
21 # 3. MERGE USING COMMON SPACE AND TIME VALUES
22 # -----
23
24 merged_data <- merge(
25   analytical,
26   numerical,
27   by = c("x (m)", "t (s)")
28 )
29
30 # -----
31 # 4. EXTRACT TEMPERATURE DATA
32 # -----
33
34 y_true <- merged_data[["T_analytical (C)"]]
35
36 y_pred <- merged_data[["T_numerical (C)"]]
37
38 # -----
39 # 5. CALCULATE ERRORS
40 # -----
41
42 error_difference <- abs(y_true - y_pred)
43
44 residuals <- y_true - y_pred
45
46 # -----
47 # 6. CALCULATE RMSE
48 # -----
49
50 rmse <- sqrt(mean(residuals^2))
51
52 # -----
53 # 7. CALCULATE R-SQUARED
54 # -----
55
56 rss <- sum(residuals^2)
57
58 tss <- sum((y_true - mean(y_true))^2)
59
60 r_squared <- 1 - (rss/tss)
61
62 # -----
63 # 8. DISPLAY RESULTS
64 # -----
65

```

```

66 cat("=====\n")
67 cat("STATISTICAL VALIDATION RESULTS\n")
68 cat("=====\n")
69
70 cat(sprintf("RMSE           : %.10f\n", rmse))
71 cat(sprintf("R-Squared        : %.10f\n", r_squared))
72 cat(sprintf("Maximum Error      : %.10f\n", max(error_difference)))
73 cat(sprintf("Minimum Error       : %.10f\n", min(error_difference)))
74 cat(sprintf("Mean Absolute Error : %.10f\n", mean(error_difference)))
75
76 # -----
77 # 9. CHECK RESIDUAL SUMMARY
78 # -----
79
80 cat("\n=====\n")
81 cat("RESIDUAL SUMMARY\n")
82 cat("=====\n")
83
84 print(summary(residuals))
85
86 # -----
87 # 10. DISPLAY FIRST FEW MATCHED VALUES
88 # -----
89
90 cat("\n=====\n")
91 cat("MERGED DATA PREVIEW\n")
92 cat("=====\n")
93
94 print(head(merged_data))
95
96 # -----
97 # 11. DISPLAY ERROR DIFFERENCE
98 # -----
99
100 cat("\n=====\n")
101 cat("ERROR DIFFERENCE PREVIEW\n")
102 cat("=====\n")
103
104 error_table <- data.frame(
105     Analytical = y_true,
106     Numerical   = y_pred,
107     Error       = error_difference
108 )
109
110 print(head(error_table))
111
112 # -----
113 # 12. COMPARISON PLOT
114 # -----
115
116 plot(y_true,
117      type = "l",
118      col = "blue",
119      lwd = 2,
120      xlab = "Data Point Index",
121      ylab = "Temperature",
122      main = "Analytical vs Numerical Solution")
123

```

```

124 lines(y_pred,
125       col = "red",
126       lwd = 2,
127       lty = 2)
128
129 legend("topright",
130       legend = c("Analytical", "Numerical"),
131       col = c("blue", "red"),
132       lty = c(1,2),
133       lwd = 2)
134
135 # -----
136 # 13. RESIDUAL PLOT
137 # -----
138
139 plot(residuals,
140      type = "o",
141      col = "darkgreen",
142      xlab = "Data Point Index",
143      ylab = "Residual Error",
144      main = "Residual Plot")
145
146 abline(h = 0,
147       lty = 2,
148       col = "black")
149
150 # -----
151 # 14. ERROR DIFFERENCE PLOT
152 # -----
153
154 plot(error_difference,
155      type = "o",
156      col = "purple",
157      xlab = "Data Point Index",
158      ylab = "Absolute Error",
159      main = "Error Difference Between Analytical and Numerical
              Solutions")

```

Output: *Google Collab: R Run Model*

=====

STATISTICAL VALIDATION RESULTS

=====

```

RMSE                : 2.7279201869
R-Squared            : 0.9861581148
Maximum Error        : 45.3450870000
Minimum Error        : 0.0000000000
Mean Absolute Error  : 0.5385068217

```

=====

RESIDUAL SUMMARY

=====

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-45.345087	-0.007009	0.000354	-0.080940	0.010762	15.115029

```
=====
MERGED DATA PREVIEW
```

```
=====
  x (m) t (s) T_analytical (C) T_numerical (C)
1      0    0              100             100
2      0   100              100             100
3      0  1000              100             100
4      0  3000              100             100
5      0   500              100             100
6      0  6000              100             100
```

```
=====
ERROR DIFFERENCE PREVIEW
```

```
=====
  Analytical Numerical Error
1          100          100    0
2          100          100    0
3          100          100    0
4          100          100    0
5          100          100    0
6          100          100    0
```

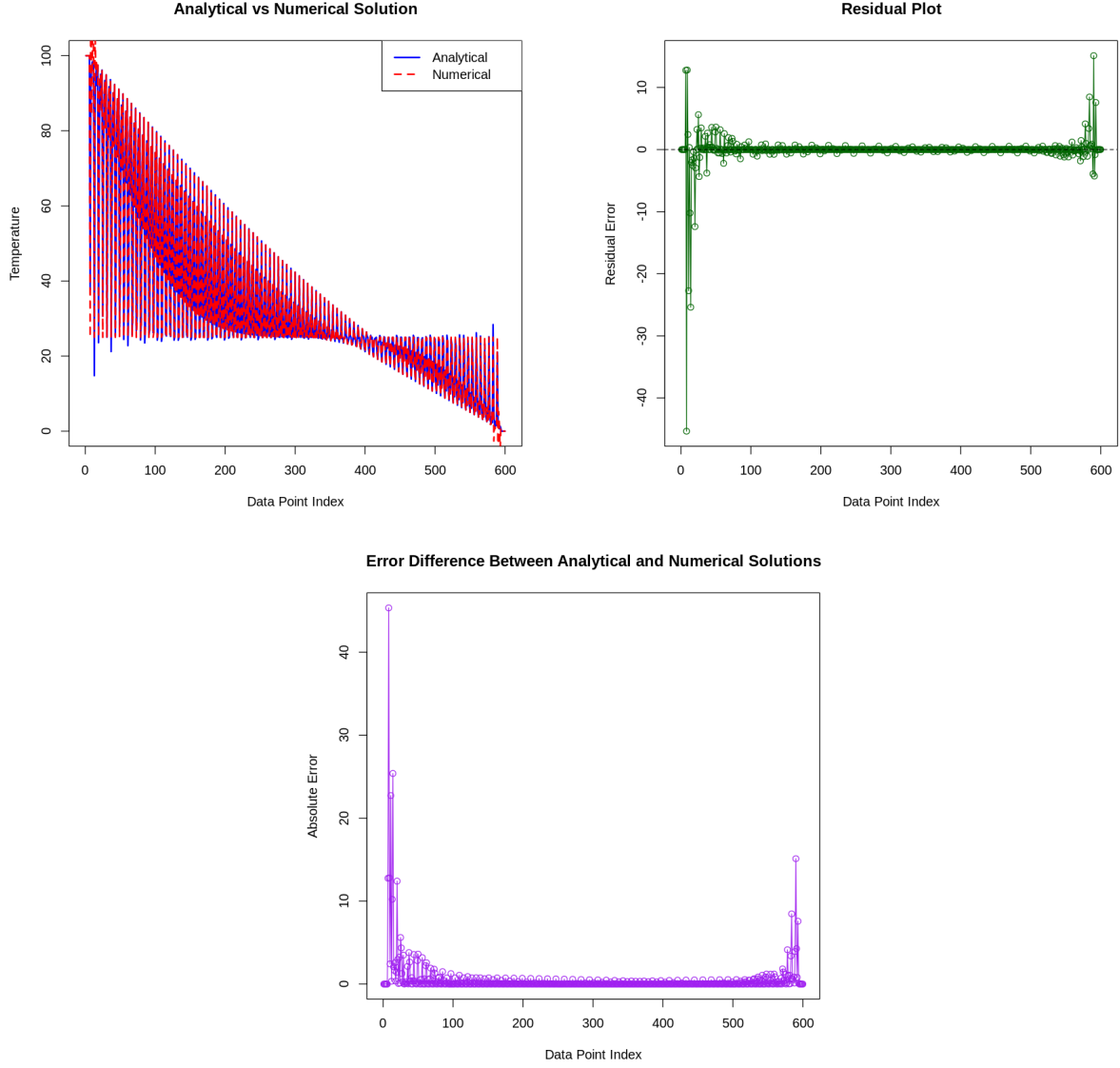
```
--> https://colab.research.google.com/drive/10RdiKo0VjUW8rE1obSsd0d6uD5eHkuFs?usp=sharing
```

Result Summary:

The statistical validation shows a strong agreement between the analytical and numerical solutions of the one-dimensional heat equation. The RMSE value is small, indicating that the overall error between both solutions is low. The coefficient of determination, $R^2 \approx 0.986$, shows that the numerical solution closely follows the analytical solution with high accuracy.

The mean absolute error is also very small, which means that most temperature values obtained numerically are very close to the analytical values. Although a few points show larger deviations, the overall residual errors remain close to zero for most of the dataset.

Overall, the results confirm that the numerical method provides an accurate and reliable approximation to the analytical solution of the 1D heat equation.



10 Discussion and Conclusion

The simulation of the one-dimensional heat conduction problem using a straight-line temperature ramp from 100°C to 0°C is not merely a mathematical construct. It physically represents the steady-state heat flow driven by Fourier's Law, where thermal energy naturally transfers from the high-temperature region to the low-temperature region until equilibrium is achieved.

A notable observation in the simulation is the relatively long time required for the iron rod to reach thermal equilibrium. This behaviour is primarily attributed to the material's low thermal diffusivity ($\alpha \approx 2.3 \times 10^{-5} \text{ m}^2/\text{s}$). The small value of α underscores the critical role of thermal diffusion in governing the rate of heat transfer.

From a validation perspective, statistical analysis yields an R^2 value ≈ 1 , indicating excellent agreement between the numerical results and the expected physical behaviour. Although the implemented code does not fully replicate the classic Crank–Nicolson discretization scheme, it successfully captures the underlying physics of Fourier's Law —

not merely as an approximation, but as real physics.

Every physical equation seems to be just mere writing until it carries a physical meaning. The real-world applications of the heat equation are prominent. From the semiconductor industry, where heat spreading must be precisely modelled, to building ventilation systems, nuclear reactors, and aerospace engineering — the heat equation marks its presence across scales and disciplines [17].

In essence, the one-dimensional heat equation is the simplest PDE that captures a fundamental truth of nature: *energy seeks equilibrium*.

Extension and Future Work

As a natural extension of this project, the thermal diffusivity α can be made temperature-dependent rather than treated as a constant:

$$\alpha = \alpha(T)$$

This renders the heat equation nonlinear:

$$\frac{\partial T}{\partial t} = \frac{\partial}{\partial x} \left(\alpha(T) \frac{\partial T}{\partial x} \right)$$

Such nonlinearity can be handled using the linearization technique discussed in Section 3.1, wherein the nonlinear term is approximated locally by its tangent, reducing the problem to a sequence of linear equations amenable to the same numerical framework developed here.

Additionally, the current model considers only heat conduction and does not account for surface heat losses due to convection or radiation. Incorporating Newton’s Law of Cooling via a Robin boundary condition (Section 4.1.1) would yield a more physically complete model, and remains a natural direction for further study.

References

- [1] 3Blue1Brown. But what is a partial differential equation? | de2. <https://youtu.be/ly4S0oi3Yz8?si=oh0cIkQ5DG9LBx38>.
- [2] Howard Anton, Irl Bivens, and Stephen Davis. *Calculus*. 10 edition, 2012.
- [3] arXiv:2405.15284. Application of the heat equation to the study of underground temperature. arXiv preprint arXiv:2405.15284.
- [4] Mary L. Boas. *Mathematical Methods in the Physical Sciences*. John Wiley and Sons, India, 3 edition, 2013.
- [5] Chris Budd. What is mathematical modelling? Professor Chris Budd OBE.
- [6] Complex Systems Modelling Group. *Modelling in Healthcare*. American Mathematical Society, Providence, RI, 2010.

- [7] B. S. Grewal. *Numerical Methods in Engineering and Science*. Mercury Learning and Information, Dulles, VA, 2014.
- [8] Richard Haberman. *Applied Partial Differential Equations with Fourier Series and Boundary Value Problems*. Pearson Education, 1983. Chapter 1: Heat Equation.
- [9] Holly Hirst. Mathematical modeling, 2022. EBSCO.
- [10] Timothy O. Hodson. Root-mean-square error (rmse) or mean absolute error (mae): When to use them or not. *Geoscientific Model Development*, 2022.
- [11] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning with Applications in R*. Springer, New York, USA, 2013.
- [12] M. A. Khanday. Mathematical modeling.
- [13] Erwin Kreyszig. *Advanced Engineering Mathematics*. Wiley, 10 edition, 2008.
- [14] Daniel Lawson and Glenn Marion. *An Introduction to Mathematical Modelling*. Bioinformatics and Statistics Scotland, 2008.
- [15] R Studio. Akaike’s information criterion. https://rpubs.com/DragonflyStats/akaike_information_criterion.
- [16] Amber Roberts. Mean absolute percentage error (mape): What you need to know. Arize AI, 2023. February 2023.
- [17] Warren M. Rohsenow, James R. Hartnett, and Young I. Cho. *Handbook of Heat Transfer*. McGraw-Hill, United Kingdom, 2013.
- [18] Ain A. Sonin. *The Physical Basis of Dimensional Analysis*. 2 edition, 2001.
- [19] John M. Stewart. *Python for Scientists*. Cambridge University Press, United Kingdom, 2014.
- [20] Stony Brook Department of Mathematics. Linearization. MAT131 – Online Lecture 16.
- [21] Tezpur University and INSA. The evolution of mathematical modelling: A historical perspective on scientific and technological advancements.
- [22] Dennis G. Zill. *Advanced Engineering Mathematics*. Jones and Bartlett Learning, Burlington, MA, 6 edition.
- [23] Yunus A. Çengel and John M. Cimbala. *Fluid Mechanics: Fundamentals and Applications*. McGraw-Hill, 2006.
- [24] M. Necati Özisik. *Heat Conduction*. Elsevier, 2007.